Complex Cascade Digital Filters

Kenneth Martin
Granite SemiCom Inc.
Toronto ON, Canada
martin@granitesemi.com

***Abstract*—Complex filters are filters that process two signals often denoted the *real* and *imaginary* signal components. This publication deals with how to realize digital complex filters using cascade architectures. A number of design techniques to improve numerical inaccuracies are presented. The design algorithms have been implemented using Matlab; methods to overcome Matlab limitations when manipulating complex systems are described. A number of examples are presented. The sensitivities of the filter realizations are investigated using Monte Carlo analysis.**

***Index Terms*—complex filters, cascade filters, pole-zero pairing, filter scaling, Monte Carlo sensitivity analysis, open-source code**

I. Introduction

A complex filter is a linear frequency-selective system that operates on complex signals and typically has poles and zeros that do not have conjugate symmetry about zero. They are useful in many communication systems for applications such as low-IF receivers and single-side-band filters.

Previously the author published techniques for finding transfer functions for band-pass complex filter approximations [3] and Matlab programs for these techniques are available at [4]. During the development of the realization programs described here-in, the approximation programs have been improved, bugs have been fixed, and algorithms have been made more numerically robust. This publication describes how digital complex filters can be realized using the bilinear transform and cascade architectures. Matlab programs used in the design of these digital cascade filters are described. The designs are limited to single-pass-band filters with low-pass filters included as special cases. In addition, real low-pass and band-pass filters can be designed as degenerate cases.

When realizing digital filters based on the bilinear-z transform, the specifications are pre-warped so that after transfer functions have been transformed, the desired specifications are met in the discrete-time domain. Assuming the pass-band is a small percentage of the sampling frequency, this results in the pre-warped specifications, used for designing the continuous-time filters, being ill-conditioned. This ill-conditioning is largely alleviated by first frequency-shifting and frequency-scaling the specifications so the pass-band of the continuous-time pre-warped filter goes from -1 rad. to 1 rad.. After the continuous-time filter has been designed, the scaling and frequency-shifting are reverted before the bilinear-z transform is applied.

Next, the complex poles and zeros are ordered and grouped into second order complex biquads, and possibly a first-order section for odd-order filters. During the grouping, the zeros are combined with the closest poles. In addition, the filters are amplitude scaled.

Matlab routines have been developed for all of these operations and are available from github.com. Mathworks has independently developed routines for designing cascade filters, but the Mathwork's routines only support *real* transfer functions, not *complex* transfer functions. Much of this publication describes the newly developed Matlab routines, and exemplify the use of Matlab for complex signal processing. Many of the routines used are coded using Matlab's object-oriented classes. Examples are used to help explain the Matlab routines. The examples include using Monte-Carlo filter simulations to investigate the filter sensitivities. The filters can also be realized using either C or Go programming languages; comparative simulation times are given for a 64-channel filter bank example.

The Matlab tool box described facilitates quick prototyping of complex filters before committing the resources required for a dedicated hardware implementation. The simulation times required for these prototypes are quite reasonable especially when realized using either C or Go.

II. A Fourth-Order Example

The first example is a fourth-order complex filter with a pass-band at positive frequencies. It has two movable transmission zeros, one in the upper stop-band and one in the lower stop-band. In addition, there is one fixed transmission zero in the lower stop-band that is chosen to end up at d.c.; this is useful to minimize any d.c. offsets such as those caused by carrier leakage induced in low-IF systems. There is also a transmission zero at infinity (for the continuous-time) prototype. The top level Matlab script for designing this filter is:

```
% csc_fltr_1_2_1: a cascade file with 1 pole at infinity ,
% 2 movable poles, and 1 fixed pole

p = [-0.25  0.25]; % initial values of movable transmission zeros
px = [-0.025]; % fixed transmission zero
% it will be at dc after frequency shift of specs
ni=1; % number of loss poles at infinity
wp(1) = -0.0125; % lower pass-band edge
wp(2) = 0.0125; % upper pass-band edge
ws = [-0.024 0.024]; % lower and upper stop-band frequencies
as = [20  20]; % attenuation goals

Ap = 0.1; % the pass-band ripple in dB
ONE_STP = 0; % treat both stop-bands as a single stop-band

% frequency shift specs so pass band is at positive frequencies
% from 0.0125 to 0.0375
```

```
[p_, px_, wp_, ws_] = shiftSpecs(p, px, wp, ws, 0.025);

% top level function for designed digital filter based on
% bilinear−z transform

H = dsgnDigitalFltr(p_, px_, ni, wp_, ws_, as, Ap, 'elliptic');
% plot resulting transfer function
[ax1, ax2] = plot_drsps(H, wp_, ws_, 'b', [−0.5 0.5 −160 1]);

% order and group poles and zeros into a cascade filter
cscdFltr = mkCscdFltrD(H, wp_);

% plot cascade filter using object function
cscdFltr.plotGn(wp_, ws_, −160, 2);

% simulate cascade filter 100 times with SS matrix elements
% varying by 0.01%
runMcCscd(cscdFltr, wp_, −160, 1e−4, 0, 100);
```

The first part of the script specifies the transmission zero numbers, initial values for the movable poles, and the specifications for the pass-band and two stop-bands. The specifications in this example are given symmetrically about dc and then shifted so the pass-band is at positive frequencies between 0.0125 and 0.0375; note the pass-band is a small fraction of the sampling frequency which is assumed to be 1.

The Matlab function dsgnDigitalFltr() is the top-level function for obtaining the discrete-time transfer function that meets the specifications; the function mkCscdFltrD() is used to design the cascade filter, and the function runMcCscd() is used to simulate 100 different realizations for their impulse responses to investigate the realization sensitivities.The resulting filter transfer functions are found using an FFT to analyze the filter impulse responses. The methods used in the top-level functions will be described subsequently. The plots of the resulting transfer functions are shown in Fig. 1.

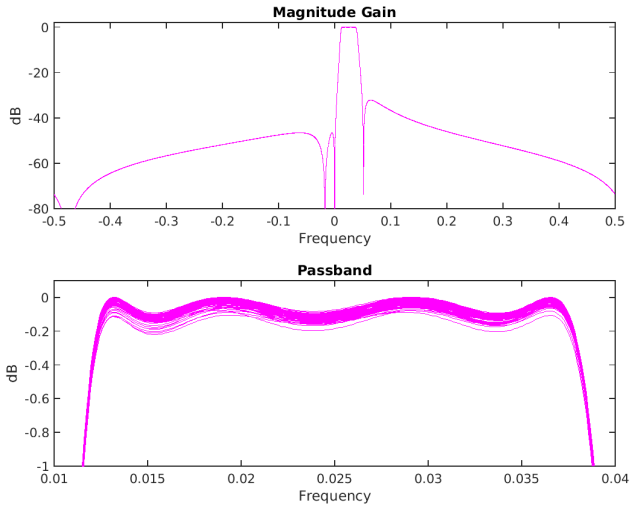

Fig. 1. 100 Monte Carlo simulations of a fourth-order complex filter.

III. The Discrete-Time Transfer Function

The discrete-time transfer function was designed using the programs from [4]. The new feature that was added was to first frequency translate the specifications to be symmetric about dc. The specifications were then pre-warped using the inverse bilinear-z transform ($\acute{f} = 2\tan(\pi f)$). Next, the specifications were frequency-scaled so the pass-band was between -1 rad. and +1 rad. These transformed specifications were used to design the continuous-time transfer function. The code for these operations is straight-forward and is not shown. The transfer function obtained was then scaled back to the original specifications, and then the discrete-time transfer function was obtained using the bilinear-z transform. The resulting filter was still symmetric about d.c. This was frequency shifted back to the desired pass-band frequencies using the following code:

```
[z,p,k,T] = zpkdata(H, 'v');
eshft = exp(j*2*pi*delta_f*T);
p2 = p*eshft;
z2 = z*eshft;
M = length(z) − length(p);
k = k*eshft^M;
H2 = zpk(z2, p2, k, T);
```

In applying the bilinear transform, Matlab's function bilinear() from the Signal-Processing Toolbox was used as this handles complex poles and zeros, unlike the bilin() function in the Control Toolbox.

IV. The Cascade Filter-Design

The first step of designing the cascade filter is to group the poles and zeros. The grouping is done using the function groupZerosD(); the Matlab code for this function is:

```
function [z3 p3] = groupZerosD(H, wp)
%
% groupZeros() is used to pair the zeros of a discrete−time transfer
% function with poles
%

  warning('off', 'Control: ltiobject:TFComplex');
  warning('off', 'Control: ltiobject:ZPKComplex');

  tol = 1e−4;
  grps = {};
  [z,p,k] = zpkdata(H, 'v');
  nz = length(z);
  np = length(p);

  % sort poles and zeros in ascending order based on angles

  z2 = sortArry(@(x) angle(x), z);
  p2 = sortArry(@(x) angle(x), p);

  % group zeros to closest poles going from zeros closest to midWp
  i = 1;
  midWp = 2*pi*(wp(1) + wp(2))/2;
  while length(z2) ~=0
    [z2 zr indx] = chooseX(@(x) (abs(angle(z2) − midWp)), z2);
    [p2 pl indx] = chooseX(@(x) (abs(angle(p2) − angle(zr))), p2);
    grps{i} = {zr, pl};
    z3(i) = zr;
    p3(i) = pl;
    i = i+1;
  end

  % for the bilinear transform, note that the number of
  % eros equals the number of poles
```

The sorting uses an anonymous function angle() which is passed into a function sortArry() along with an array of the zeros (or poles); internally in sortArry(), the Matlab function sort() is used change the order of the zeros (or poles) into

increasing order in terms of their angles. Next, each complex zero is grouped with the complex pole having the closest angle (in absolute terms). The grouping starts with zeros closest to the middle of the pass-band and then works its way to zeros further from the pass-band middle based on the magnitudes of the angles. After the first-order complex zeros have been grouped with closest poles, then every two zero-pole group is combined to make a complex biquad; this is done in the function mkCscdFltr().

Matlab supports object-oriented classes; two of these classes are used in the Cascade Filter toolbox. One of the these is used to encapsulate each biquad and assorted manipulation functions; the second is used to abstract the complete biquad filter and includes sections for each biquad. The class for biquad sections is called cscFltrSctnClass. The data stored for each section is: the array of zeros, the array of poles, the gain constant, the order (either 1 or 2), the sampling frequency (currently always 1), and the Matlab zpk() model. The methods for the cascade sections are: cscFltrSctnClass() for instantiating a biquad, setSys() for setting the zpk model, setZ() for setting the zeros, setP() for setting the poles, setK() for setting the gain constant, getGain() for finding the peak amplitude gain (in the l-∞ norm sense), setGain() for setting the peak amplitude gain, freqEval() for evaluating the transfer function at specified frequency points, sim() for simulation the biquad using a state-space difference equation, and disp() for displaying the biquad section.

The class for a cascade filter is called cascadeClass. The data stored for this class is: an array of biquad sections (of the class cscFltrSctnClass just described), the number of sections, and a zpk model for the over-all cascade transfer function. The methods of the cascadeClass are: cascadeClass() for instantiating a member of cascadeClass, addSctn() for adding an additional biquad section to the cascade filter, freqEval() for evaluating the frequency response of the cascade filter at specified frequencies, scaleFltr() for scaling all the sections of the cascade filter so that the peak gains from the cascade filter input to the biquad filter outputs are all unity (in the l-∞ norm), getSystem() for updating the internal zpk model as the product of the transfer functions of the biquad sections, and returning the over-all system model, fsclFltr() for frequency shifting the cascade filter by a specified amount, plotGn() for plotting the gain of the system model of the cascade filter, and sim() for simulating the cascade filter based on difference equations for each biquad.

V. Monte-Carlo Sensitivity Analysis

The cascade filters sensitivities are investigated using Monte-Carlo analysis. For this publication, this investigation involved simulating the impulse response of 100 perturbed cascade filters. Each impulse response was done by simulating the particular cascade filter for 8192 input samples; the first sample was one, the rest were all zeros. The impulse responses were then analyzed using an FFT, and the magnitude of the individual FFT's were then plotted. The built-in function for the cascade object class is the simplest way to do the simulation (xout = cscdFltr.sim(xin, fShft)), but given the large number of sample points (8192 times 100), a faster alternative is to simulate without using the object functions; this alternative approach is based on using Matlab's cell arrays to store the models for each biquad. The simulation times on a relatively older computer was 14 seconds versus 17 seconds (14us per time step vs 17us per time step ignoring the time for the FFT's). The code for the difference equation simulations for each biquad is:

```
function xout = simBiquad(A_, B_, C_, D_, xin, delta_f)

  eshft = exp(j*2*pi*delta_f);
  npts = length(xin);
  xout = zeros(npts, 1);

  nmbSctns = size(A_, 2);
  for k = 1:nmbSctns
    a = A_{k};
    b = B_{k};
    c = C_{k};
    d = D_{k};

    N = size(a, 1);
    X = zeros(N, 1);

    for i = 1:npts
      xout(i) = c*X + d*xin(i);
      X1 = a*X + b*xin(i);
      X = X1*eshft;
    end
    xin = xout;
  end
```

The author's opinion is these fast simulation times are remarkable considering Matlab is an interpreted language.

VI. An Example with Large Stop-Band Attenuation

A more difficult example that illustrates the ability of the toolbox for designing filters with large stop band attenuation has the following top-level code:

```
% initial guess at finite loss poles
p = [-0.25 -0.1 -0.08 -0.06 0.08 0.25];
ni=1; % number of loss poles at infinity
wp = []; ws = [];
wp(1) = -0.025; % lower pass-band edge
wp(2) = 0.025; % upper pass-band edge
ws = [-0.49 -0.05 0.05 0.49];
as = [70 50 50 70];
Ap = 0.05; % the pass-band ripple in dB
px = [];

[p_, px_, wp_, ws_] = shiftSpecs(p, px, wp, ws, 0.05);
```

This example has the responses shown in Fig. 2.

The simulation time for 100 Monte Carlo runs of 8192 time samples, and analyzing the outputs using an FFT took only 25 seconds on a moderately old desktop computer. The standard deviations for the system matrices of each biquad was 2e-5; the accuracy with no coefficient deviations was exceedingly high; the simulated pass-band very accurately matched its ideal ripple of 0.05dB. Finding a filter realization's transfer function by plotting the FFT of the simulated impulse response was more accurate than expected.

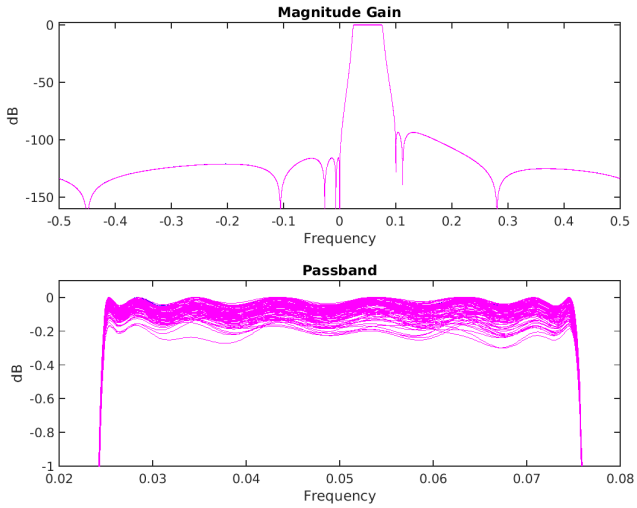

Fig. 2. Monte-Carlo simulations of a complex filter having large stop-band attenuation.

VII. A 64-Channel Filter Bank Example

Frequency scaling digital filters is very simple; it requires just a single complex multiplication directly before or after each delay block (see Fig. 19 of [8]). This makes realizing filter banks of IIR band-pass filters simple. The simulation portion of the function for realizing a 64-channel IIR filter bank, with each filter being 6'th order complex (equivalent to 12'th order real) consists of only 23 lines of code and is shown below:

```
for i = 1:npts
  Xin(i, :) = xin(i);
  for k = 1:nmbSctns
    a = A{k};
    b = B{k};
    c = C{k};
    d = D{k};

    if size(a, 1) == 2
      Xout(i, :) = c(1).*X1(k,:) + c(2).*X2(k,:) + d*Xin(i, :);
      Xi1(k,:) = a(1,1).*X1(k,:) + a(1,2).*X2(k,:) + b(1).*Xin(i, :);
      Xi2(k,:) = a(2,1).*X1(k,:) + a(2,2).*X2(k,:) + b(2).*Xin(i, :);
      X1(k,:) = Xi1(k,:).*wshft;
      X2(k,:) = Xi2(k,:).*wshft;
    else
      Xout(i, :) = c(1).*X1(k,:) + d*Xin(i, :);
      Xi1(k,:) = a(1,1).*X1(k,:) + b(1).*Xin(i, :);
      X1(k,:) = Xi1(k,:).*wshft;
    end
    Xin(i, :) = Xout(i, :);
  end
  a=1;
end
```

The simulation of the frequency responses of all 64 channels for an 8192 impulse input signal are shown in Fig. 3. Note the stop-band attenuation is greater than 100dB. The plot of the FFT's added together, except for channel 8, is shown as the red trace. The reason for not including channel 8 is to demonstrate the ability of the filter bank to "notch-out" an interfering signal. This large attenuation also allows for applications where a pilot tone (used for fast timing acquisition) is sent on one channel and can subsequently be removed. The filter bank was also simulated using the Go programming language and the C programming language. The simulation times varied; their approximate times (on a Lenovo laptop) for 8192 samples for all 64 channels were: Matlab: 480ms, Go: 44ms, C: 24ms[1]. We hope to investigate using this filter bank (in Go) on the Raspberry Pi for ECG analysis.

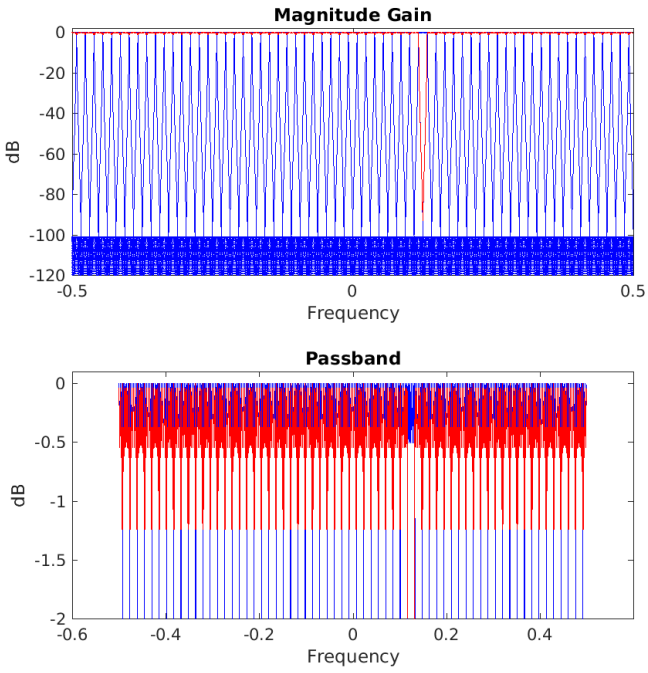

Fig. 3. FFT's of 64-channel filter-bank impulse response simulations. Red trace shows FFT of outputs added together except for channel 8.

VIII. Conclusion

Matlab routines for realizing cascade complex filters have been described. The examples demonstrate excellent stop-bands. The filters can be easily realized using Go or C in embedded Linux single-board computers.

References

[1] Introduction to Circuit Synthesis and Design, G.C. Temes and W. LaPatra, McGraw Hill, 1977.

[2] A.S. Sedra and P.O. Brackett, Filter Theory and Design: Active and Passive, Matrix Publishers, 1978.

[3] K. Martin, Approximation of Complex IIR Bandpass Filters without Arithmetic Symmetry, *IEEE Trans. Circuits and Systems I*, Vol. 52, Issue 4, pp. 794-803, April 2005.

[4] `https://github.com/kwmartin/KM_ComplexFilterToolbox`.

[5] W.M. Snelgrove, A.S. Sedra, G.R. Lang, and P.O. Brackett, Complex Analog Filters, 1981, unpublished, available at https://github.com/kwmartin/KM_ComplexFilterToolbox/blob/master/doc/snelgrove_cmplx.pdf.

[6] B.M. Silveira, C. Ouslis, and A.S. Sedra, Passive Ladder Synthesis in filtor X, *IEEE Intl. Symp. On Circuits and Systems*, pp. 1431-1434, 1993.

[7] R. Burden and J.D. Faires, Numerical Analysis,, Fourth Ed., Alg. 2.7, pp. 79-81, PWS-Kent, 1989, and coding by John Matthews, MatlabCentral/FileExchange.

[8] K. Martin, "Complex Signal Processing is Not Complex", *IEEE Trans. Circuits and Systems 1*, Vol. 51, No. 9, pp. 1823-1836, Sept. 2004.

[1]C was compiled with the -ffast-math flag; without this flag, C simulated simulated much slower: 90ms. Simulating on the Raspberry Pi 3B+: Go: 68ms, C (with -ffast-math): 280ms; we do not know why C with complex math is so slow on the Pi; Go is much faster (120kS/s for all 64 channels).